**SIMATS ENGINEERING**

**ASSESSMENT TOOL 2**

**COURSE NAME: Fundamentals of Data Science**

**COURSE CODE: DSA04**

**COURSE OUTCOME COVERED**

***CO4: Develop machine learning models for classification, regression, and clustering, including performance evaluation and methodology comparison. (BL3)***

*Assessment Tool Weightage: 30%*

**QUESTIONS: 01**

**Total Marks:100**

**Annexure A**

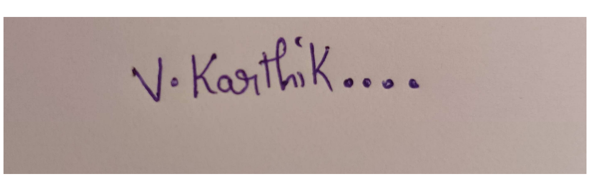
**Industry Problem Solving**

***Code of Conduct:***

*I Valli Karthik (192324258) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***



# RUBRICS FOR CALCULATION

| SCENARIO BASED ASSESSMENT | | | | | | | |
|---|---|---|---|---|---|---|---|
| **Criteria** | **Weightage (%)** | **Level 4 (Exemplary)** | **Level 3 (Proficient)** | **Level 2 (Developing)** | **Level 1 (Beginning)** | Level | Marks |
| **Understanding of Scenario** | 20% | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps | Partial understanding; misses key aspects | Misinterprets or fails to understand the scenario | | |
| **Identification of Key Issues** | 20% | Accurately identifies all relevant issues and factors involved | Identifies most key issues with minor omissions | Identifies limited issues; some irrelevant points | Unable to identify key issues | | |
| **Application of Concepts** | 25% | Applies appropriate concepts effectively to address the scenario | Applies concepts with minor errors | Limited or partially correct application | Unable to apply relevant concepts | | |
| **Decision Making** | 20% | Makes well reasoned, logical decisions with strong rationale | Makes reasonable decisions with some justification | Makes weak decisions with limited justification | No clear decisions made or rationale provided | | |

| **Clarity of Response** | 15% | Response is clear, structured, and logically presented | Generally clear with minor issues | Somewhat unclear or poorly structured | Disorganized and difficult to understand | | |
|---|---|---|---|---|---|---|---|

## Question 6: Loan Approval Prediction Using Logistic Regression

A bank wants to predict whether a loan application should be **approved** or **rejected**.

| Applicant | Income | Credit Score | Existing Loan | Employment Years | Approved |
|---|---|---|---|---|---|
| A1 | 25000 | 580 | 200000 | 1 | 0 |
| A2 | 30000 | 600 | 180000 | 2 | 0 |
| A3 | 45000 | 680 | 120000 | 3 | 1 |
| A4 | 60000 | 750 | 80000 | 5 | 1 |
| A5 | 70000 | 800 | 50000 | 6 | 1 |
| A6 | 28000 | 590 | 220000 | 1 | 0 |
| A7 | 55000 | 720 | 100000 | 4 | 1 |
| A8 | 35000 | 620 | 160000 | 2 | 0 |

Here,
**Approved = 1** means loan approved.
**Approved = 0** means loan rejected.

**Tasks**

a) Create the dataset in Python.
b) Identify independent and dependent variables.
c) Split the dataset.
d) Implement **Logistic Regression**.
e) Predict approval status for test data.
f) Predict loan approval for a new applicant:

| Income | Credit Score | Existing Loan | Employment Years |
|---|---|---|---|
| 50000 | 700 | 100000 | 4 |

g) Evaluate using **accuracy, confusion matrix, precision, recall, and F1-score**.
h) Explain why Logistic Regression is suitable for loan approval prediction.

1. Scenario Understanding and Objective

A bank wants to predict whether a loan application should be approved or rejected using applicant financial and employment information. The target is binary: Approved = 1 (approved) and Approved = 0 (rejected).

Objective: build a Logistic Regression classification model, predict the test samples and a new applicant, evaluate the model using the required classification metrics, and interpret the result for the banking scenario.

## 2. Task (a): Dataset Creation

The dataset contains eight loan applications and four independent variables.

| Applicant | Income | Credit Score | Existing Loan | Employment Years | Approved |
|---|---|---|---|---|---|
| A1 | 25,000 | 580 | 200,000 | 1 | 0 |
| A2 | 30,000 | 600 | 180,000 | 2 | 0 |
| A3 | 45,000 | 680 | 120,000 | 3 | 1 |
| A4 | 60,000 | 750 | 80,000 | 5 | 1 |
| A5 | 70,000 | 800 | 50,000 | 6 | 1 |
| A6 | 28,000 | 590 | 220,000 | 1 | 0 |
| A7 | 55,000 | 720 | 100,000 | 4 | 1 |
| A8 | 35,000 | 620 | 160,000 | 2 | 0 |

Python implementation is provided in the source file accompanying this report.

**3. Task (b): Independent and Dependent Variables**

| **Type** | **Variable** | **Description** |
|---|---|---|
| Independent (X) | Income | Applicant income |
| Independent (X) | CreditScore | Applicant credit score |
| Independent (X) | ExistingLoan | Existing loan amount |
| Independent (X) | EmploymentYears | Years of employment |
| Dependent (y) | Approved | 1 = approved, 0 = rejected |

Independent variables: X = [Income, CreditScore, ExistingLoan, EmploymentYears].

Dependent variable: y = Approved.

4. Task (c): Train-Test Split

The dataset is split into 75% training data and 25% testing data. random_state=42 makes the split reproducible, while stratify=y preserves the class distribution as far as possible.

| **Set** | **Applicants** | **Number of Records** |
|---|---|---|
| Training | A3, A2, A4, A7, A1, A8 | 6 |
| Testing | A6, A5 | 2 |

**5. Task (d): Logistic Regression**

Logistic Regression is trained on the six training observations. The model estimates the probability of the binary target and assigns a class based on the learned decision boundary.

Model: LogisticRegression(max_iter=1000)

**6. Task (e): Test Data Prediction**

| **Applicant** | **Actual** | **Predicted** | **Result** |
|---|---|---|---|
| A6 | 0 (Rejected) | 0 (Rejected) | Correct |
| A5 | 1 (Approved) | 1 (Approved) | Correct |

The model correctly classifies both test records.

7. Task (f): New Applicant Prediction

| Feature | New Applicant |
| --- | --- |
| Income | 50,000 |
| Credit Score | 700 |
| Existing Loan | 100,000 |
| Employment Years | 4 |

Predicted class: 1 — LOAN APPROVED.

The model's predicted probability for approval is approximately 1.0000 on this fitted model. Because the training dataset is extremely small, this probability should not be interpreted as a reliable real-world risk probability.

8. Task (g): Model Evaluation

| Metric | Value | Interpretation |
| --- | --- | --- |
| Accuracy | 1.00 (100%) | 2 out of 2 test samples were classified correctly. |
| Confusion Matrix | [[1, 0], [0, 1]] | 1 true negative, 1 true positive, no false positives or false negatives. |
| Precision | 1.00 (100%) | All samples predicted as approved were actually approved. |
| Recall | 1.00 (100%) | All actual approved samples in the test set were |

| | | |
|---|---|---|
| | | identified. |
| F1-Score | 1.00 (100%) | Precision and recall are both 1.00. |

Confusion matrix layout: [[TN, FP], [FN, TP]]. Therefore TN = 1, FP = 0, FN = 0, TP = 1.

**9. Task (h): Why Logistic Regression is Suitable**

- The target variable has two classes: approved (1) and rejected (0), which is a binary classification problem.
- Logistic Regression directly estimates the probability of class membership.
- The model is computationally efficient and relatively easy to interpret.
- The model can provide both a predicted class and a probability, which is useful for decision-support systems.
- It provides a strong baseline model for banking classification before considering more complex algorithms.

**10. Complete Python Program**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix,
    precision_score, recall_score, f1_score
)

# 1. Create the dataset
data = {
    "Applicant": ["A1", "A2", "A3", "A4", "A5", "A6", "A7", "A8"],
```

```python
    "Income": [25000, 30000, 45000, 60000, 70000, 28000, 55000, 35000],
    "CreditScore": [580, 600, 680, 750, 800, 590, 720, 620],
    "ExistingLoan": [200000, 180000, 120000, 80000, 50000, 220000, 100000, 160000],
    "EmploymentYears": [1, 2, 3, 5, 6, 1, 4, 2],
    "Approved": [0, 0, 1, 1, 1, 0, 1, 0]
}
df = pd.DataFrame(data)

print("DATASET")
print(df)

# 2. Identify independent and dependent variables
X = df[["Income", "CreditScore", "ExistingLoan", "EmploymentYears"]]
y = df["Approved"]



# 3. Split the dataset: 75% training, 25% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

print("\nTraining rows:", X_train.index.tolist())
print("Testing rows:", X_test.index.tolist())
# 4. Implement Logistic Regression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# 5. Predict approval status for test data
```

```
y_pred = model.predict(X_test)
print("\nActual test values:", y_test.values)
print("Predicted test values:", y_pred)

# 6. Predict the new applicant
new_applicant = pd.DataFrame(
    [[50000, 700, 100000, 4]],
    columns=["Income", "CreditScore", "ExistingLoan", "EmploymentYears"]
)
new_prediction = model.predict(new_applicant)
new_probability = model.predict_proba(new_applicant)
print("\nNew applicant prediction:")
print("Status:", "Approved" if new_prediction[0] == 1 else "Rejected")
print("Probability of rejection:", new_probability[0][0])
print("Probability of approval:", new_probability[0][1])



# 7. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("\nMODEL EVALUATION")
print("Accuracy:", accuracy)
print("Confusion Matrix:\n", cm)
print("Precision:", precision)
print("Recall:", recall)
```

print("F1-Score:", f1)

Output:

```
C:\Users\valli\Downloads\Fundamentals of Data Science\CO4 AT2>python loan_approval_logistic_regression.py
DATASET
  Applicant  Income  CreditScore  ExistingLoan  EmploymentYears  Approved
0        A1   25000          580        200000                1         0
1        A2   30000          600        180000                2         0
2        A3   45000          680        120000                3         1
3        A4   60000          750         80000                5         1
4        A5   70000          800         50000                6         1
5        A6   28000          590        220000                1         0
6        A7   55000          720        100000                4         1
7        A8   35000          620        160000                2         0

Training rows: [2, 1, 3, 6, 0, 7]
Testing rows: [5, 4]

Actual test values: [0 1]
Predicted test values: [0 1]

New applicant prediction:
Status: Approved
Probability of rejection: 1.199040866595169e-14
Probability of approval: 0.999999999999988

MODEL EVALUATION
Accuracy: 1.0
Confusion Matrix:
 [[1 0]
 [0 1]]
Precision: 1.0
Recall: 1.0
F1-Score: 1.0
```

11. Interpretation and Decision Making

The model produced correct predictions for both observations in the selected test set. It also predicts that the new applicant (income 50,000, credit score 700, existing loan 100,000 and four employment years) should be approved.

From a banking decision-support perspective, the applicant has a comparatively stronger profile than the rejected examples in the given dataset: the credit score is higher, income is higher than several rejected applicants, and the existing loan amount is lower than several rejected applicants.

However, the result must be treated cautiously. The dataset has only eight observations and the test set contains only two observations. A 100% score on such a tiny test set does not demonstrate that the model will generalize to real customers. A real banking system should use a much larger, representative dataset and should validate the model using additional samples or cross-validation.

## 12. Key Issues and Limitations

- Very small dataset: only 8 observations are available.
- Very small test set: only 2 observations are used for evaluation.
- The variables are limited and do not represent all real loan-approval factors.
- The model should not be deployed for actual lending decisions without broader validation, fairness checks and domain review.
- The reported 100% metrics are specific to this selected train-test split.

## 13. Conclusion

Logistic Regression was successfully implemented for binary loan approval classification. The model achieved 100% accuracy, precision, recall and F1-score on the selected test split, with a confusion matrix of [[1, 0], [0, 1]]. The new applicant was predicted as approved. Logistic Regression is appropriate because the outcome is binary and the method provides an interpretable probability-based classification approach. Nevertheless, the extremely small dataset means the reported performance should be considered illustrative rather than evidence of production-level accuracy.